

A Hybrid Genetic Algorithm for Minimizing Total Flowtime in Flowshop Scheduling Problems

曾怜玉^{1,2} 林亞泰¹

¹ 中興大學資訊科學與工程學系

² 中興大學資訊網路與多媒體研究所

lytseng@cs.nchu.edu.tw, ted6622@ms17.hinet.net

摘要

近年來以最小化 total flowtime 為目標的 flowshop scheduling problem 愈來愈受到學者的重視並加以研究。本研究提出 hybrid genetic algorithm 來解決此問題，此演算法以基因演算法做為全域搜尋並且依據基因演算法的收斂程度，採用兩種強弱不同的區域搜尋演算法：Insertion Search (IS)與 Insertion Search with Cut-Repair (ISCR)，不但改進了解的品質更增加了搜尋的效率。實驗的結果顯示，hybrid genetic algorithm 的效能非常好，與近幾年發表的期刊論文相較，在相同的 90 個 benchmark problem 中它更新了 59 個目前已知的最佳解。

關鍵字：基因演算法、flowshop scheduling problem、total flowtime

一、簡介

Flowshop Scheduling Problem 是一個重要的排程問題，自從 Johnson 於 1954 年提出此問題後，就引起許學者注意並加以研究。一般而言，Flow-Shop Scheduling Problem 的目標在於找出一個排程，使得完成所有工作所花費的時間最小，一般稱為 makespan。近年來，應因生產模式的改變，有些學者也開始注意到另一種準則：total flowtime，或稱為 total completion time，因為 total flowtime 愈小代表在製品的數量愈少，所以它直接影響到後勤作業的效能[8]。

Flowshop Scheduling Problem 的定義如下：存在 n 個工作 $\{J_1, J_2, \dots, J_n\}$ 與 m 個機器 $\{M_1, M_2, \dots, M_m\}$ 。每個工作包含 m 個需要依序在不同機器上加工的操作，所有工作的加工順序皆相同。每個的操作的處理時間皆為已知，以 T_{ij} ($i=1, \dots, n; j=1, \dots, m$) 來表示第 i 個工作的第 j 個操作的處理時間。另外，每部機器一次只能處理一個工作，不可中斷或搶先，且每個工作在同一時間點，只能被一台機器處理。

所謂的 flowtime 代表一個工作於最後一台機器的完成時間。給定一個排程 $\pi = \{\pi_1, \pi_2, \dots, \pi_n\}$ ，則每個工作的完成時間可被遞迴地計算：

$$C(\pi_j, k) = \max\{C(\pi_{j-1}, k), C(\pi_j, k-1)\} + T_{\pi_j, k}, \quad j = 2, \dots, n; k = 2, \dots, m$$

where $C(\pi_0, k) = 0$ and $C(\pi_j, 0) = 0$ for relevant j and k

而此問題的目標，就是找一個排程使得 n 個工作的 flowtime 總合最小，一般表示為：

$$C_{sum} = \sum_{j=1}^n C(j, m)$$

這個問題的定義看似簡單，但是當 $m > 2$ 時，則是一個 NP-hard 問題[4]，因此，許多學者發展了一些啟發式演算法(Heuristic algorithm)希望在有限的時間內可以找到好的解，這些方法大致可分兩類：建構式啟發式演算法(Constructive heuristic)，例如：Wang 等學者(1997)[17]、Woo 與 Yim (1998)[16]以及改進式啟發式演算法(Improvement heuristics)，例如：Ho (1995)[6]。

另外，Rajendran 與 Ziegler (1997)[10]則是提出兩階段的方法，在執行完建構式啟發式演算法後，再執行改進式啟發式演算法，進一步改進所找到的解。Liu 與 Reeves (2001)[8]對這些方法做了比較，同時也提出一個新的建構式啟發式演算法。此方法的特色在於，利用動態索引函式(Dynamic index function)用來決定建構過程中下一個要排入的工作，而動態索引函式在設計上不但考慮加入某個工作之後對目前已排好的排程之影響，也考慮到對其他未排入排程的工作的影響。演算法的最後再執行區域搜尋以求得更好的解。在與其他啟發式演算法方法，如：Ho (1995) [5]、Rajendran 與 Ziegler (1997)[10]、Wang 等學者 (1997)[17]、Woo 與 Yim (1998)[16]等做比較後 Liu 與 Reeves 的方法得到較好的結果。

Allahverdi 與 Aldowaisan (2002)[1]比較了現有的幾種方法，最後以效果最好的方法為基礎，加入 pair-wise exchange procedure，將 7 種不同組合的啟發式演算法進行比較，最後提出 IH7 heuristic。Framinan 與 Lesten(2003)[2]所提出的 FL heuristic 則是在建構階段的每一回合中對目前所產生的部份解做區域搜尋(Local Search)，其效果雖然比 IH7 差，但若是以此方法取代 IH7 的初始階段則可以得到更好的結果。最近，Framinan、

Lesten 與 R. Ruiz-Usano (2005)[3]將近年來所有著名的啟發式演算法進行分類與比較後發現，FL heuristic 所產生的結果優於其他方法。

啟發式演算法的優點是速度快，但往往無法找到最佳解。因此，最近有許多學者運用 meta-heuristic 的方法來解決此問題。Rajendran 與 Ziegler (2004)[11]提出兩個以螞蟻演算法(Ant Approach)為基礎的方法：MMAS 與 PACO。而 Tasgetiren 等學者 (2005)[14]則是提出以粒子群優化(Particle Swarm Optimization, PSO)結合 Variable Neighborhood Search 做為區域搜尋的方法。三者皆實驗於相同的 90 個 benchmark problem 上做比較，Tasgetiren 等學者 (2005)[14]所提出的 PSO 在效能上，優於上述的兩種螞蟻演算法。

本研究將針對 flowshop scheduling problem 以最小化 total flowtime 為目標，提出一個 Hybrid Genetic Algorithm。此演算法以基因演算法做為全域搜尋並結合 Insertion Search 與 Insertion Search with Cut-and-Repair 做為區域搜尋。此方法，與上述的 MMAS、PACO、PSO 測試於相同的 benchmark problem 上，結果不論是在品質與效率上都有很好的表現。

本論文之架構如下：第二節介紹我們所提出之 Hybrid Genetic Algorithm 與兩種區域搜尋方法；接著在第三節說明我們的實驗結果。最後，第四節將說明本論文之貢獻與未來研究工作。

二、混合式基因演算法(Hybrid Genetic Algorithm)

本章節將介紹本研究所出的 Hybrid Genetic Algorithm，並利用它來解決以最小化 total flowtime 為目標的 Flowshop Scheduling Problem。我們以基因演算法扮演全域搜尋的角色，對整個解空間做搜尋。Insertion Search (IS) 與 Insertion Search with Cut-and-Repair (ISCR)則是混合在基因演算法中，依據基因演算法的收斂程度決定使用的時機，從實驗結果可以看出這樣的組合所帶來的效益。另外，本研究提出以直交表(Orthogonal Array)為基礎設計的 Orthogonal Array Crossover (OA-Crossover)以增進基因演算法的效能。在此，我們先描述 Hybrid Genetic Algorithm 的整體架構，有關 Orthogonal Array Crossover 與兩種區域搜尋的演算法將後續的章節做介紹。Hybrid Genetic Algorithm 的架構如下：

/* initialization */

Step 1. Set the values of the population size (P_s), the crossover rate (P_c), the mutation rate (P_m), the termination condition (Max_Stuck) and the threshold for local search ($Local_Stuck$). Set $S_l = 0$.

Step 2. Produce the initial population that consists of P_s randomly generated chromosomes.

Step 3. Evaluate the total flowtime of each chromosome in the population. Deposit the chromosome with the best total flowtime in $BEST$ and its total flowtime in C^* .

/* crossover, local search and selection */

Step 4. Repeat Step 5 to Step 7 $P_s \times P_c$ times.

Step 5. Randomly choose two chromosomes P_1 and P_2 . Apply OA-crossover to the parent chromosomes P_1 and P_2 to produce the child chromosome $Child$.

Step 6. If $S_l < Local_Stuck$ then apply Insertion Search to $Child$.

Otherwise, apply Insertion Search with Cut-and-Repair to $Child$.

Step 7. P_1 and P_2 are replaced by the best two of P_1 , P_2 and $Child$.

/* mutation */

Step 8. Randomly choose $P_s \times P_m$ chromosomes from the population and apply the mutation operator to these chromosomes.

Step 9. Find the chromosome B with the best total flowtime C in the population. If $C < C^*$ then $C^* \leftarrow C$, $BEST \leftarrow B$ and $S_l \leftarrow 0$. Otherwise, $S_l \leftarrow S_l + 1$.

Step 10. If $S_l > Max_Stuck$ then stop. Otherwise, go to Step 4.

上述的演算法中，Step 5 到 Step 7 會執行所提出的 OA-Crossover 與區域搜尋共 $P_s \times P_c$ 次，其中 P_s 代表基因演算法的族群大小(Population size)，而 P_c 代表交配率(Crossover rate)，OA-Crossover 將在後面的 2.2 節做介紹，兩種區域搜尋的方法也將在之後的 2.3 節說明。接著，執行 Step 8 的突變(Mutation)共 $P_s \times P_m$ 次， P_m 代表突變機率。從 Step 4 到 Step 9 可以被視為一個基因演算法的世代(generation)。在 Step 6 中，若 S_l 小於 $Local_Stuck$ 則對子代執行 Insertion_Search (IS)。但若 S_l 大於 $Local_Stuck$ 時，則對子代執行 Insertion Search with Cut-and-Repair (ISCR)。先使用 Insertion Search (IS)再使用 Insertion Search with Cut-and-Repair (ISCR)的原因在於，IS 有可能落入區域最佳解(local optimum)而 ISCR 能搜尋較大的鄰近區域(neighborhood)可能會幫助我們跳出區域最佳解。最後，若沒進步的代數大於 Max_Stuck 則停止搜尋。

2.1 基因演算法的編碼與適應函數的定義

在基因演算法中，染色體(chromosome)用來代表在解空間(solution space)中的一組解(solution)。針對 Flowshop Scheduling Problem，本研究以工作的排列 π (permutation)做為染色體的編碼。舉例而言，以一個包含 6 個工作的 flowshop scheduling problem，若有一個染色體為 permutation $\pi = [2, 3, 1, 6, 5, 4]$ ，就代表這 6 個工作

的執行順序為 $J_2, J_3, J_1, J_6, J_5, J_4$ 。另外，本研究直接依染色體的工作順序進行排程，所得的 total flowtime 就用來做為基因演算法的適應值(fitness value)。

2.2 OA-Crossover

於本節中，我們將介紹 Orthogonal Array(OA) 使用在實驗設計上之概念，若需要了解更多細節可以參考[9]。說明完 Orthogonal Array 後，再介紹本研究提出的使用在基因演算法中之 OA-Crossover operator。

假設一個實驗中，有 k 個 factor，每個 factor 都有 q 個 level，若要進行完全因子實驗 (full-factorial experiment) 找到每個因子最好的設定，需要進行 q^k 次的實驗。一般而言，進行這樣大規模的實驗需要耗費相當大的成本。因此，我們需要一個數量較少，同時又具代表性的實驗組合，而 Orthogonal Array 就是為此目的而發展出來的方法。使用 Orthogonal Array 來組合實驗因子進行部份因子實驗 (fractional-factorial experiment) 並配合因子分析就可以推得各因子的影響力與其 level 的最佳設定。Orthogonal Array 是一個有 n 列與 k 欄的矩陣，每個元素的值介於 0 到 $q-1$ 之間，其表示法為： $OA(n, k, q, t)$ 或是 $L_n(q^k)$ ，其中 n 代表實驗的次數； k 代表因子 (factor) 的個數； q 代表因子的 level 數，另外，可對 array 設定一個強度參數 (strength) t ，代表 array 中任 t 個欄位所組成的 sub-matrix 中所有可能組合的出現機率相同。舉例而言：在一個強度為 3，level 為 2 的 OA 中，任三個欄位中「000」、「001」、「010」、「011」、「100」、「101」、「110」、「111」這 8 種組合的出現機率相同，於第二種表示法中強度參數 (strength) t 省略，代表其設定為 2。以表 1 為例：假設一個有 7 個因子，每個因子都有兩個 level 的實驗，我們以一個 $L_8(2^7)$ 的 Orthogonal Array 來進行取樣並觀察實驗結果進行因子分析。令 E_i 為第 i 次實驗的評估函數值；第 j 個因子的 level 值為 k 時的影響力 F_{jk} 定義為：

$$F_{jk} = \sum_{i=1}^n E_i \times A_{ijk}$$

其中 A_{ijk} 代表，若第 i 次的實驗中第 j 個因子選用的 level 為 k ，則 A_{ijk} 設為 1，否則為 0。計算的結果，若實驗設計上評估函數是愈大愈好的話，則當 $F_{j0} > F_{j1}$ 就代表第 j 個因子的 level 設為 0 對評估函數的貢獻度較大，反之，若 $F_{j0} < F_{j1}$ 則代表 level 設定為 1 較佳。

表 1. $L_8(2^7)$ orthogonal array.

Test No.	Factors							Evaluation Value (E_i)
	1	2	3	4	5	6	7	
1	0	0	0	0	0	0	0	E_1
2	0	0	0	1	1	1	1	E_2
3	0	1	1	0	0	1	1	E_3
4	0	1	1	1	1	0	0	E_4
5	1	0	1	0	1	0	1	E_5
6	1	0	1	1	0	1	0	E_6
7	1	1	0	0	1	1	0	E_7
8	1	1	0	1	0	0	1	E_8

Tsai 等學者 (2004)[15]曾提出以 OA 為基礎的 Crossover 來解決 global numerical optimization problem。Ho 與 Chen (2000)[6]也提出相同的設計應用於 traveling salesman problem。本研究以相似的概念設計了 OA-Crossover 來找出較好的子代。Orthogonal Array Crossover (OA-Crossover)的做法如下所述：

- Step 1. Let N be the number of pieces into which the user wants to cut parent chromosomes P_1 and P_2 for recombination. Generate the orthogonal array $L_{N+1}(2^N)$.
- Step 2. Randomly choose parent chromosomes P_1 and P_2 . Randomly choose $N-1$ cut points to cut P_1 and P_2 into N subsequences.
- Step 3. Consult the i th row of the OA $L_{N+1}(2^N)$ and generate a sampled child C_i , for $i=1, 2, \dots, N+1$. The j th subsequence of C_i is taken from the j th subsequence of P_1 if the level of the j th factor in row i of the OA is 0. Otherwise, the j th subsequence of C_i is taken from the j th subsequence of P_2 . Repair C_i whenever it is necessary. (Repair scheme will be described later.)
- Step 4. Calculate the evaluation value E_i of each sampled child C_i , for $i=1, 2, \dots, N+1$. The evaluation value E_i is the fitness value (i.e. the reciprocal of the total flowtime) of the chromosome C_i .
- Step 5. Calculate the main effect F_{jk} of factor j with level k , for $j=1, 2, \dots, N$ and $k=0, 1$.
- Step 6. Find the best level for each factor. The best level of factor j is k if $F_{jk} = \max\{F_{j0}, F_{j1}\}$. Use the best levels of all factors to generate another child C_{N+2} . Repair C_{N+2} whenever it is necessary. Calculate the evaluation value E_{N+2} .
- Step 7. From $C_1, C_2, \dots, C_{N+2}$, choose the one with the best evaluation value to be the child chromosome.

在 Step 3 與 Step 6 中產生的子代 C_i 與 C_{N+2} 有可能需要調整為合理解，調整的方式以圖 1 為例加以說明。圖 1 中包含一個 $L_4(2^3)$ 的 Orthogonal Array 以及參與 crossover 的 parent 1 與 parent 2。今假設隨機產生的切點(cut point)為 2 與 4，以產

生 C_2 為例，要參考的是 Orthogonal Array 中的第二列 011。因為第一個因子的 level 為 0，所以選擇由 parent 1 的 sub-sequence 繼承 Job 到子代中；第二個因子的 level 為 1，所以子代繼承自 parent 2 的 sub-sequence；最後一個因子也是 1，所以也是從 parent 2 繼承，但是 Job 5 與 Job 3 已經出現過了，因此必需留下空白。最後，尋找尚未排入染色體中的 Job 2 與 Job 6，並依照它們在 Parent 1 之中的順序，填入之前留下的空白，因此最後兩個位置為 Job 2 與 Job 6。

2.3 區域搜尋演算法

區域搜尋(Local search)的目的是希望從一組解的鄰域(neighborhood)中找到更好的解。一組解 π 在解空間(solution space)中可被視為一個點， π 的 neighborhood $N(\pi)$ 定義為：對 π 執行一個操作(operator)後可到達的所有點所形成之集合。經常被用於本問題的操作(operator)的有交換操作(exchange operator)與插入操作(insertion operator)。交換操作表示為 $Exch(x, y)$ ，代表將位於 x 的 job 與在位置 y 的 job 交換，而插入操作表示為 $Ins(x, y)$ ，代表將位於 x 的 job 插入在位置 y 。針對這兩種操作做比較後發現，使用插入操作的效果優於交換操作，因此本研究採用插入操作做為區域搜尋的鄰域(neighborhood)。由於染色體的長度為工作的個數 n ，採用插入操作共有 $n*(n-1)$ 種組合，經由實驗的結果發現，若將所有的組合都加以測試，從所花費的時間與品質改善的比率來看效果並不好，因此我們在所提出的 Insertion Search (IS) 中加入了一個控制搜尋範圍的變數 α ，以增加方法的彈性與效率。為方便說明我們以 π 代表一組解， $\pi(i)$ 代表 π 中第 i 個位置的 job，所提出的 Insertion Search 架構如下：

Procedure Insertion-Search (π, c, α)

- Step 1. Set the search List $SL \leftarrow \{1, 2, \dots, n\}$.
- Step 2. If SL is not empty, randomly choose p from SL , remove p from SL , and go to Step 3. Otherwise, stop and return π and C .
- Step 3. Execute insertion operators $Ins(p, k)$ for $k=p-1, p-2, \dots, \max(p-\alpha, 1)$ and $p+1, p+2, \dots, \min(p+\alpha, n)$. Calculate the total flowtime after each insertion operator and choose the best one. If the total flowtime of the best solution is better than C , let π be the best solution and C be the total flowtime of the best solution and go to step 1. Otherwise, go to step 2.

從實驗中發現，在基因演算法中加入 Insertion Search 做為區域搜尋後，確實能有效改善求解的品質，但有時候會落入區域最佳解，而使基因演算法逐漸同質化。這是因為 Insertion Search 所搜尋的鄰域(neighborhood)比較小所致，所以本研究提出另一個區域搜尋稱為 Insertion Search with Cut-and-Repair (ISCR)，結合 Insertion Search 與 Cut-and-Repair procedure 搜尋較大的鄰域，以增加求得解的多樣性(diversification)。為方便說明，在此先介紹 Cut-and-Repair procedure，再說明整個 Insertion Search with Cut-and-Repair (ISCR) 的演算法。Cut-and-Repair procedure 的想法是從目前的排程中，找出相鄰且造成較多閒置時間(idle time)的一對工作(Job)，將它們從中切斷，然後找其他造成閒置時間較小的工作插入該位置。閒置時間 $IT_{\pi(j)}$ 的定義為：給定一個 schedule π ， $\pi(j)$ 代表 π 中第 j 個位置的工作，則相鄰的兩個工作之間的閒置時間定義如下：

$$IT_{(\pi(j), \pi(j+1))} = \sum_{x=2}^m \max\{C(\pi(j+1), x-1) - C(\pi(j), x), 0\},$$

where $j=1, 2, \dots, n-1$

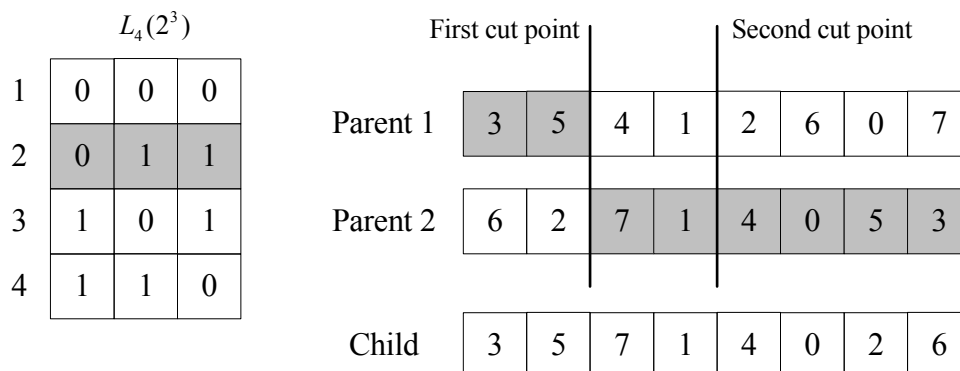


圖 1. Orthogonal Array Crossover 範例

Cut-and-Repair procedure 架構如下：

Procedure Cut-and-Repair (π, c)

- Step 1. Set Cutting List $CL \leftarrow \emptyset$ and Moving List $ML \leftarrow \emptyset$.
- Step 2. Compute $IT_{(\pi(j), \pi(j+1))}$ for $j=1, 2, \dots, n-1$. Choose the pairs of adjacent jobs that has the largest and the second largest IT values. Put the two cut points in CL .
- Step 3. If CL is not empty, randomly choose one cut point $(cp, cp+1)$ from CL and go to Step 4. Otherwise, stop and return π and C .
- Step 4. For $t \in \{1, 2, \dots, cp-1\}$, execute $Ins(t, cp)$. For $t \in \{cp+2, cp+3, \dots, n\}$, execute $Ins(t, cp+1)$. From above insertion operators, find the eight operators that results in the largest amount of idle time reduction. Put these eight operators in ML .
- Step 5. Generate a random number θ in $[0, 1)$. If θ is greater than 0.5, choose the best operator in ML ; otherwise, randomly choose an operator from ML . Apply this operator to π and update C accordingly. Set $ML \leftarrow \emptyset$ and go to Step 3.

接著說明本研究所提出的 Insertion Search with Cut-and-Repair，其架構如下：

- Step 1. $\pi^* \leftarrow \pi, c^* \leftarrow c$ and $iter \leftarrow 0$.
- Step 2. Execute Insertion-Search (π, c, α) with return values in π and c . If $c < c^*$ then $\pi^* \leftarrow \pi$ and $c^* \leftarrow c$.
- Step 3. Execute Cut-and-Repair (π, c) with return values in π and c . $\pi^* \leftarrow \pi$ and $c^* \leftarrow c$.
- Step 4. $iter \leftarrow iter + 1$. If $iter \geq Max_Loop$ then stop and return π^* and c^* . Otherwise, go to Step 2.

2.4 突變(Mutation)

適當的突變(Mutation)可以提供多樣性以避免陷入區域最佳解。由於 2.3 節的區域搜尋採用插入操作(Insertion operator)，為提昇搜尋的多樣性，因此我們以交換操作(Exchange operator)做為突變的機制。隨機從基因演算法的群體(population)中選擇 $P_s \times P_m$ 個染色體，對每個染色體執行 $Exch(x, y)$ 共 t 次， x 與 y 為隨機選擇的位置且 $1 \leq t \leq T$ ， T 可由使用者決定。

三、實驗結果

本研究以 C++ 實作所提出的 Hybrid Genetic Algorithm，測試的平台為個人電腦，使用的 CPU 為 AMD K7 1.83GHz，記憶體為 512MB，作業系統為 windows XP。為了比較 Hybrid Genetic Algorithm 與其他方法，包括 Liu 與 Reeves (2001)[8]、Rajendran 與 Ziegler (2004)[11] 以及 Tasgetiren 等學者 (2005)[14] 等，在效能上之差異。因此，在實驗的部份，我們採用與上述學者相同的 benchmark problems 做測試，此 benchmark problems 是由 Taillard [13] 所提供。表 2 為本實驗中 Hybrid Genetic Algorithm 的參數設定，而與各種方法的比較的結果列在表 3。

在表 3 中，Min 代表各種方法所能找到的最小的 total flow time，LR 為 Liu and Reeves (2001) [8] 在文獻中用來做為比較標準的解，這些解都是以啟發式方法(heuristic)所能找到最好的解，但都不是由同一種啟發式方法產生。MMAS 與 PACO 則是由 Rajendran 與 Ziegler (2004) [11] 所提出的兩種 Ant Colony Algorithms。PSO 則是最近 Tasgetiren 等學者 (2005) [14] 所提出的 Particle Swarm Optimization Algorithm，而最後一欄 Hybrid GA 為本研究所提出的 Hybrid Genetic Algorithm。其中 Hybrid Genetic Algorithm 的數據是根據表 2 的設定，對每個問題執行了 10 次的獨立測試而得到的結果，其中以粗體字表現的部份代表所得到的最佳解優於其他方法，於 90 題之中共計 59 題。

於 CPU 時間方面，由於時間的花費與問題的大小成正比，因此，我們將大小相同的問題分組平均，將結果與 PSO 方法做的比較，結果如表 4 所示。PSO 測試平台為 Intel P4 2.6GHz 的個人電腦，擁有 256MB 的記憶體，在實驗的平台相差不多的情況下，我們的方法在時間與品質上皆有不錯的表現。

表 2. Hybrid Genetic Algorithm 之參數設定

	P_s	P_c	P_m	Max_Stuck	Local_Stuck	OA	α_1	α_2	Max_Loop
n=20	20	50%	5%	10	5	$L_8(2^7)$	10	10	2
n=50	100	50%	5%	10	5	$L_{16}(2^{15})$	10	25	2
n=100	100	50%	5%	10	8	$L_{16}(2^{15})$	10	50	2

表 3. Hybrid Genetic Algorithm 與其他方法之比較

n*m	LR	MMAS	PACO	PSO	Hybrid GA		n*m	LR	MMAS	PACO	PSO	Hybrid GA	
	Min	Min	Min	Min	Min	Avg		Min	Min	Min	Min	Min	Avg
20,5	14226	14056	14056	14033	14033	14037.8	50,20	129095	127348	126962	128622	126073	126448.3
	15446	15151	15214	15151	15151	15180.4		122094	121208	121098	122173	119300	119737
	13676	13416	13403	13301	13301	13328.7		121379	118051	117524	118719	116856	117194.8
	15750	15486	15505	15447	15447	15475.3		124083	123061	122807	123028	121028	121404.4
	13633	13529	13529	13529	13529	13529		122158	119920	119221	121202	118736	118943
	13265	13139	13123	13123	13123	13123		124061	122369	122262	123217	121066	121516.3
	13774	13559	13674	13548	13548	13586.3		126363	123609	125351	125586	123580	123879.3
	13968	13968	14042	13948	13948	13964.7		126317	124543	124374	125714	122770	123175.7
	14456	14317	14383	14295	14295	14319.6		125318	124059	123646	124932	121872	122364.3
	13036	12968	13021	12943	12943	12969.4		127823	126582	125767	126311	124354	124849.6
20,10	21207	20980	20958	20911	20911	20925.1	100,5	256789	257025	257886	254762	254619	255198.4
	22927	22440	22591	22440	22440	22459.8		245609	246612	246326	245315	243817	244825
	20072	19833	19968	19833	19833	19845.7		241013	240537	241271	239777	239075	239697.2
	18857	18724	18769	18710	18710	18751.4		231365	230480	230376	228872	228291	228787
	18939	18644	18749	18641	18641	18661.9		244016	243013	243457	242245	241255	241742.3
	19608	19245	19245	19249	19245	19294.6		235793	236225	236409	234082	233583	234260.1
	18723	18376	18377	18363	18363	18364.3		243741	243935	243854	242122	241458	241930.4
	20504	20241	20377	20241	20241	20255.9		235171	234813	234579	232755	232283	232813
	20561	20330	20330	20330	20330	20330		251291	252384	253325	249959	249269	250235.7
	21506	21320	21323	21320	21320	21329.5		247491	246261	246750	244275	243879	244647.4
20,20	34119	33623	33623	34975	33623	33719	100,10	306375	305004	305376	303142	300634	302291.8
	31918	31604	31597	32659	31587	31590.7		280928	279094	278921	277109	277209	278049.9
	34552	33920	34130	34594	33920	33925		296927	297177	294239	292465	290198	291703.2
	32159	31698	31753	32716	31661	31691.1		309607	306994	306739	304676	303669	305501.3
	34990	34593	34642	35455	34557	34587.3		291731	290493	289676	288242	287136	288222.6
	32734	32637	32594	33530	32564	32570.1		276751	276449	275932	272790	273172	273951.3
	33449	33038	32922	33733	32922	32989.9		288199	286545	284846	282440	281306	283139
	32611	32444	32533	33008	32412	32429.4		296130	297454	297400	293572	293628	294827.2
	34084	33625	33623	34446	33600	33611.9		312175	309664	307043	305605	304276	305295.3
	32537	32317	32317	33281	32262	32271.6		298901	296869	297182	295173	293465	295056.2
50,5	65663	65768	65546	65058	64853	64924.6	100,20	383865	373756	372630	374351	370603	371741.8
	68664	68828	68485	68298	68173	68263.2		383976	383614	381124	379792	375982	377991.6
	64378	64166	64149	63577	63367	63524		383779	380112	379135	378174	373554	375336.2
	69795	69113	69359	68571	68281	68522.1		384854	380201	380765	380899	376236	378454.3
	70841	70331	70154	69698	69551	69670.8		383802	377268	379064	376187	373524	374314.3
	68084	67563	67664	67138	67013	67120		387962	381510	380464	379248	374705	377063.5
	67186	67014	66600	66338	66294	66405.5		384839	381963	382015	380912	376998	378533.3
	65582	64863	65123	64638	64560	64635.8		397264	393617	393075	392315	388058	389421.6
	63968	63735	63483	63227	63029	63190.1		387831	385478	380359	382212	378474	380306.2
	70273	70256	69831	69195	69037	69187		394861	387948	388060	386013	383283	384028.7
50,10	88770	89599	88942	88031	87599	87783.5							
	85600	83612	84549	83624	83001	83312							
	82456	81655	81338	80609	80224	80453.4							
	89356	87924	88014	87053	86787	87050.8							
	88482	88826	87801	87263	86646	86910.4							
	89602	88394	88269	87255	86826	87003							
	91422	90686	89984	89259	88996	89371.1							
	89549	88595	88281	87192	86860	87345.6							
	88230	86975	86995	86102	85841	86104.3							
	90787	89470	89238	88631	88293	88578.9							

表 4. Hybrid Genetic Algorithm 與 PSO 的平均 CPU 時間之比較

n×m	PSO	Hybrid GA
20×5	3.18	0.11
20×10	7.21	0.22
20×20	11.93	0.39
50×5	41.71	16.55
50×10	74.49	40.33
50×20	143.32	82.23
100×5	222.28	94.17
100×1	407.88	251.62
100×2	824.41	588.86

四. 結論與未來工作

本研究針對 flow-shop scheduling problem 以最小化 total flow-time 為目標，提出一個有效的 Hybrid Genetic Algorithm。此演算法以基因演算法為基礎，結合兩種強弱不同的區域搜尋演算法：Insertion Search (IS) 與 Insertion Search with Cut-Repair (ISCR)。從基因演算法開始執行，直到演算法收斂的初期，我們使用的是簡單且快速的 Insertion search 做為區域搜尋，使其能廣泛地在解空間(solution space)中的各個區域進行的搜尋，以增加整個方法求解的多樣性(diversification)。而當基因演算法收斂到一定程度，再使用搜尋能力較強的 Insertion Search with Cut-Repair (ISCR)，一方面可以將這些可能存在最佳解的區域做更精細的搜尋，另一方面也可以使其避免落入區域最佳解，這樣的結合不但增加了求解的品質同時也相對地具有效率。實驗的結果顯示，hybrid genetic algorithm 的效能非常好，與近幾年發表的期刊論文相較，在相同的 90 個 benchmark problem 中它更新了 59 個目前已知的最佳解。未來我們希望能將此概念應用在其他排程問題上，例如：Job shop scheduling problem 與 Open shop scheduling problem。

五. 致謝

本研究承蒙國科會計劃 (NSC 96-2628-E-0058-074-MY3) 之資助，特此致謝。

參考文獻

- [1] Allahverdi, A., Aldowaisan, T., 2002. New heuristics to minimize total completion time in m-machine flowshops. *International Journal of Production Economics* 77, 71-83.
- [2] Framinan, J.M., Leisten, R., 2003. An efficient constructive heuristic for flowtime minimisation in permutation flow shop. *OMEGA*, 31, 311-317.
- [3] Framinan, J.M., Leisten, R., Ruiz-Usano, R., 2005. Comparison of heuristics for minimisation in permutation flowshops. *Computers & Operations Research* 32, 1237-1254.
- [4] Garey, M.R., Johnson, D.S., Sethi, R., 1976. The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research* 1, 117-129.
- [5] Ho, J.C., 1995. Flowshop sequencing with mean flow time objective. *European Journal of Operational Research* 81, 571-578.
- [6] Ho, S.Y., Chen, J.H., 2000. A GA-based systematic reasoning approach for solving traveling salesman problems using an orthogonal array crossover. in *Proceeding of The Fourth International IEEE Conference/Exhibition on High Performance Computing in Asia-Pacific Region*, Beijing, China, pp. 659-663.
- [7] Johnson, S.M., 1954. Optimal two-and-three stage production schedules with setup times included. *Naval Research Logistics Quarterly* 1, 61-68.
- [8] Liu, J., Reeves, C.T., 2001. Constructive and composite heuristic solutions to the $P//\sum C_i$ scheduling problem. *European Journal of Operational Research* 132, 439-452.
- [9] Montgomery, D.C., 1991. *Design and Analysis of Experiments*, 3rd Ed. New York: Wiley.
- [10] Rajendran, C., Ziegler, H., 1997. An efficient heuristic for scheduling in a flowshop to minimize total weighted flowtime of jobs. *European Journal of Operational Research* 103, 129-138.
- [11] Rajendran, C., Ziegler, H., 2004. Ant-colony algorithms for permutation flowshop scheduling to minimize makespan/total flowtime of jobs. *European Journal of Operational Research* 155(2), 426-438.
- [12] Rajendran, C., Ziegler, H., 2005. Two ant-colony algorithms for minimizing total flowtime in permutation flowshops. *Computer & Industrial Engineering* 48, 789-797.
- [13] Taillard, E., 1990. Some efficient heuristic methods for the flow shop sequencing problem. *European Journal of Operations Research* 47, 65-74.
- [14] Tasgetiren, M.F., Liang, Y.-C., Sevkli, M., Gencyilmaz, G., 2007. A particle swarm optimization algorithm for makespan and total flowtime minimization in the permutation flowshop sequencing problem. *European Journal of Operational Research* 177, 1930-1947.
- [15] Tsai, J.T., Liu, T.K., Chou, J.H., 2004. Hybrid Taguchi-genetic algorithm for global numerical optimization. *IEEE Transactions on Evolutionary Computation* 8(4), 365-377.
- [16] Woo, D.S., Yim, H.S., 1998. A heuristic

- algorithm for mean flowtime objective in flowshop scheduling. Computers and Operational Research 25, 175-182.
- [17] Wang, C., Chu, C., Proth, J.-M., 1997. Heuristic approaches for $n/m/F//\sum C_i$ scheduling problems. European Journal of Operational Research 96, 636-644.
- [18] Yamada, T., Reeves, C.R., 1998. Solving the C_{sum} permutation flowshop scheduling problem by genetic local search. In: Proceedings of 1998 IEEE International Conference on Evolutionary Computation, pp. 230-234.